

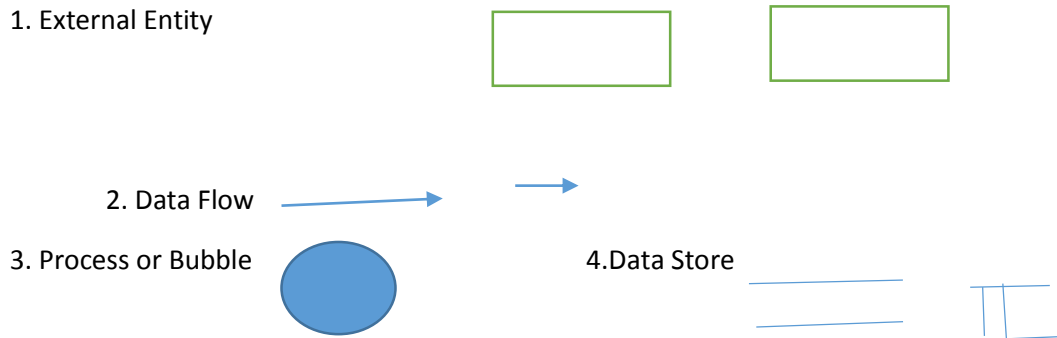
Unit 2 DFD (Data Flow diagram): DFD stands for Data Flow Diagram. It is also known as Bubble chart through which we can represent the flow of data graphically in an information system.

-By using DFD we can easily understand the overall functionality of system because diagram represent the incoming data flow and stored data in graphical form.

-It describes how data is processed in a system in term of input and output.

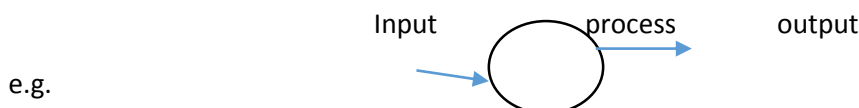
-A DFD model use a number of notations or symbol to represent flow of data.

Entity is a real world object or concept about which you want to store information .An entity can be place ,person,object or concept.



Rules of DFD:

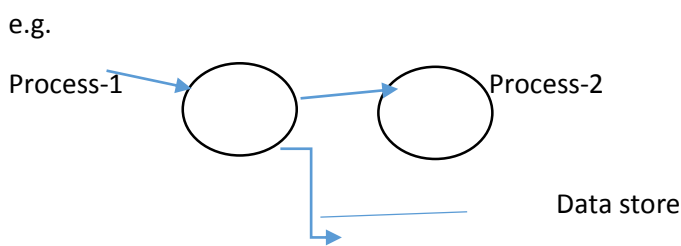
1. Each process should have at least one input and one output.



2. Each data store should have at least one data flow in and one data flow output.

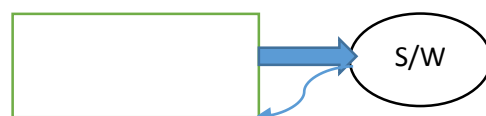


3. All process in a DFD go to either another process or data store.

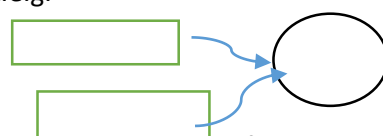


4. All the external entities must be connected through a process and Entity can provide something to the software as well as entity can consume some data from the software.

Levels of DFD:

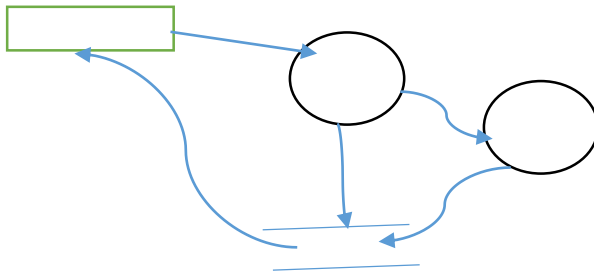


1) **0 th Level DFD:** It is a diagram which provides the entire system, data flows and processing with a single process (bubble) is called as context.e.g.

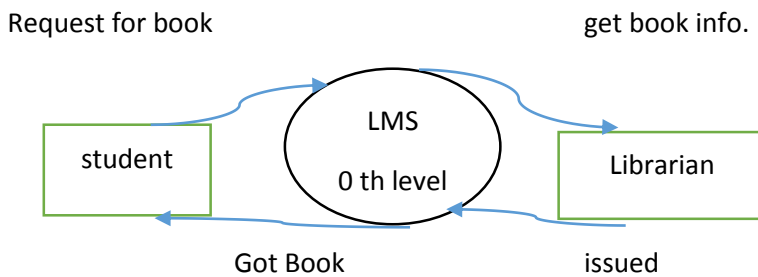


2) **1 st level DFD:** This is the more detailed version of the previous level that includes the database and various important units.

E.g.



Example-Create a DFD for Library Management system.



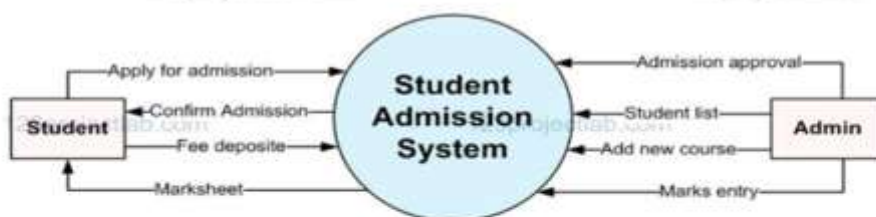
A **Zero Level Data Flow Diagram (DFD)**, also known as **Level 0 DFD**, is a high-level representation of a system's data flow. It gives a bird's-eye view of how data moves through the system, without going into much detail. It shows the entire system as a single process, interacting with external entities and data stores.

Level 1 DFD (Detailed Breakdown of Major Processes)

A **Level 1 DFD** breaks down the major process shown in the **Zero Level DFD** into smaller, more detailed sub-processes. It shows the key functions that the system performs and how they interact with external entities and data stores.

Level 2 DFD (Even More Detailed Breakdown)

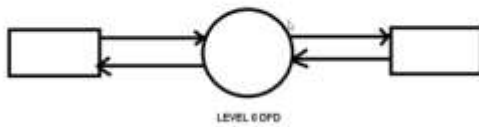
A **Level 2 DFD** further decomposes the processes from the **Level 1 DFD** into even more specific sub-processes. It adds more detail about how each process works internally.



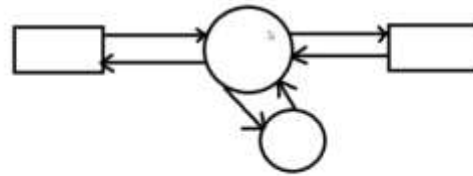
Context Level DFD for Student Admission System

Level 0 DFD for Student Admission System

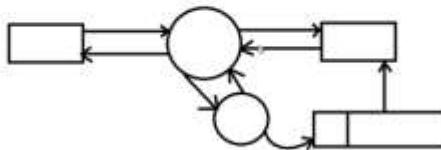
LEVEL 0 DFD



LEVEL 1 DFD



Level 2 DFD



Basic Elements of DFD / Components of DFD

□ Symbols used in designing the DFD are:

SYMBOL NAME	SYMBOL	MEANING
Circle	○	Process
Open rectangle	⌊	Data store
Rectangle	▭	Entity
Arrows	↔	Data flow

Types of DFD

Logical DFD

- It shows how data flows in a system.
- It describes what data is moved from one entity to another.
- It focus on what happens in the data flow.
- It shows how system works not how the system can be implemented.
- It is implementation independent.

Physical DFD

- It shows how data flow is actually implemented in the system.
- It describes how data is moved from one entity to another.
- It shows how the current system operates and how a system will be implemented.
- It is implementation dependent.

ERD: -ERD stands for Entity Relationship Diagram. An **Entity-Relationship Diagram (ERD)** is a visual representation of the relationships between different entities in a system. It helps in designing and understanding the structure of a database by showing how data entities are related to each other

-The diagram generated by E-R model is known as ERD.

-ERD shows the relationship of different entities stored in the database.

Figures	Symbol	Representation
Rectangle		Entities in the ER Model
Ellipse		Attributes in the ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types



Represents Entity



Represents Attribute

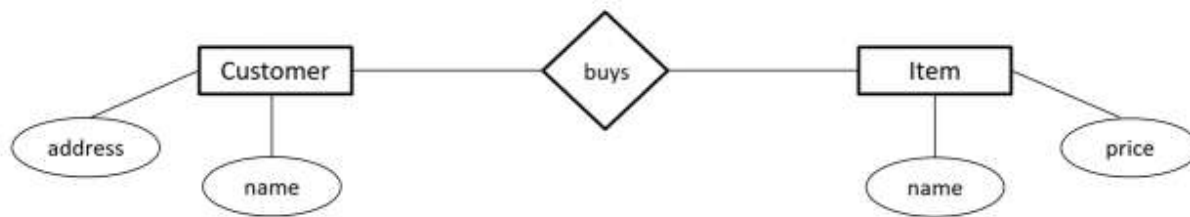


Represents Relationship



Links Attribute(s) to entity set(s) or
Entity set(s) to Relationship set(s)

What is ERD



Customer

Name	Address
Rohan	12/ABC
Sohan	15/XYZ

Item

Name	Price
mouse	125
keyboard	150

What is ERD

- ❑ ER diagram explains the logical structure of database.
- ❑ At last, an ERD is implemented as a database.
- ❑ It helps us to analyze data requirements systematically to produce a well-designed database.
- ❑ It is considered as a best practice to complete ER modeling before implementing your database.

ERD : What is ERD ?

What is ERD

❑ Entity:

- ❑ An entity is a real world object or concept about which you want to store information. An entity can be place, person, object or a concept etc.

❑ Example

- ❑ Person: student, employee, patient
- ❑ Place: building, store
- ❑ Object: car, bike, product
- ❑ Concept: course

- ❑ It is represented using rectangle shape.

Student

Car

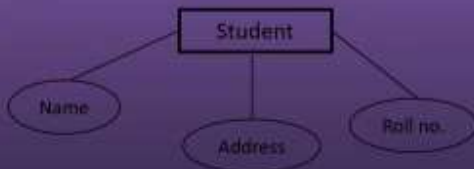
Course

Store

What is ERD

❑ Attributes:

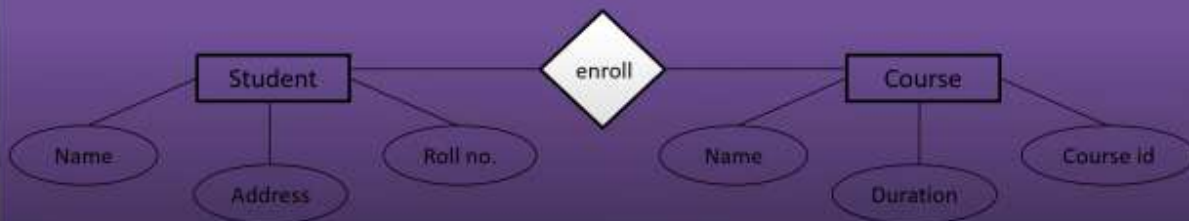
- ❑ An attribute are the properties or characteristics of an entity.
- ❑ It describes an entity.
- ❑ It is represented using oval shape.



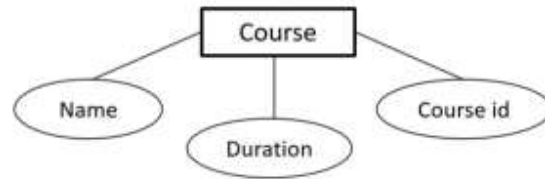
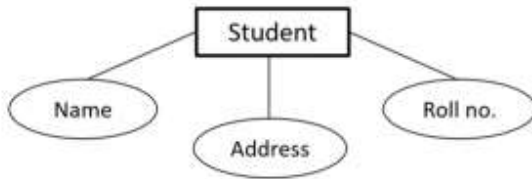
What is ERD

❑ Relationship:

- ❑ It specifies how entities are associated with each other.
- ❑ It is represented using diamond shape.



What is ERD



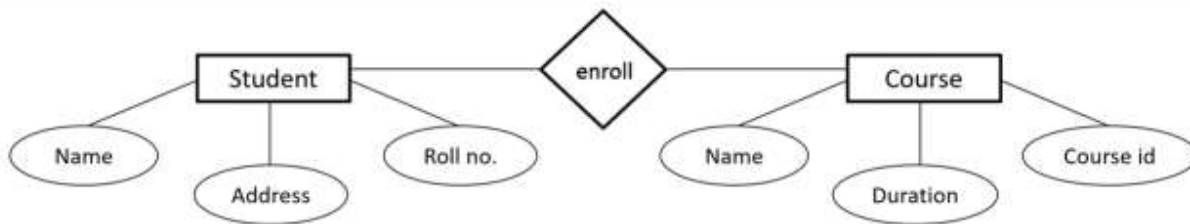
Student

Name	Address	Roll No.
Rohan	12/ABC	001
Sohan	15/XYZ	002

Course

Name	Duration	Course id
Core java	125	101
Advance java	150	102

What is ERD



Student

Name	Address	Roll No.
Rohan	12/ABC	001
Sohan	15/XYZ	002

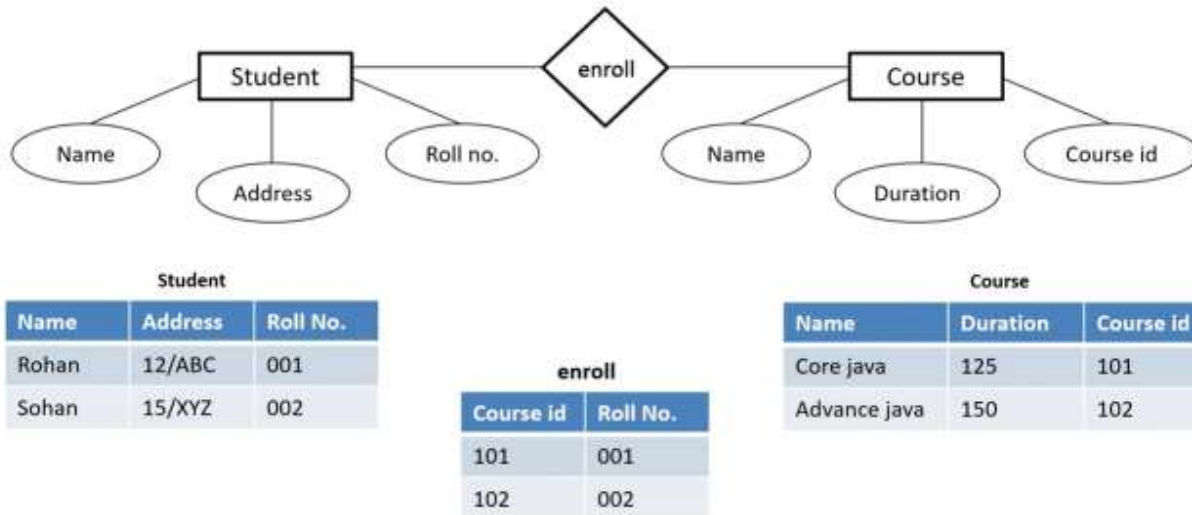
enroll

Course id	Roll No.
101	001
102	002

Course

Name	Duration	Course id
Core java	125	101
Advance java	150	102

What is ERD



Explanation of the Diagram:

1. Entities:

- Student**, **Course**, and **Professor** are the main entities.
- Each has its own set of **attributes** (like Student_ID, Name, etc.).

2. Relationships:

- Student ↔ Enrollment ↔ Course:** This many-to-many relationship is handled by the **Enrollment** entity. Each student can enroll in multiple courses, and each course can have multiple students.

Key Points to Remember:

- Entity:** Represents a table in the database (e.g., Student, Course).
- Attributes:** Characteristics or properties of an entity (e.g., Student_ID, Name).
- Relationships:** Show how entities are connected (e.g., a Student enrolls in many Courses).

In an **Entity-Relationship Diagram (ERD)**, attributes describe the properties of an entity. There are different types of attributes used in database design. Here's a simple explanation with examples:

1. Single-Valued Attribute

A **single-valued attribute** is an attribute that holds **only one value** for each entity instance.

Example:

- Student Entity:** Roll Number (each student has only one roll number)
- Car Entity:** Registration Number (each car has a unique registration number)
- Employee Entity:** Date of Birth (each employee has only one birth date)

Difference from Multi-Valued Attribute:

- Single-Valued** → One value per entity (e.g., **Age** of a person)
- Multi-Valued** → More than one value per entity (e.g., **Phone Numbers** of a person)

2. Composite Attribute

An attribute that **can be divided into smaller parts**.

Example:

- **Full Name** → First Name, Last Name
- **Address** → Street, City, State, Zip Code

3. Derived Attribute

An attribute that is **calculated from other attributes**.

Example:

- **Age** (can be derived from Date of Birth)
- **Total Price** (calculated from Quantity × Price per item)

4. Multi-Valued Attribute

An attribute that **can have multiple values**.

Example:

- **Phone Numbers** (A person may have multiple phone numbers)
- **Skills** (A job candidate can have multiple skills)

Why ERDs are Important:

- ERDs are used to visually represent **database structure**.
- They help developers **understand the relationships** between data entities.
- They are an essential part of **database design** to ensure that the system works smoothly and that the data is organized correctly.

What is a Data Dictionary?

A **data dictionary** is a tool that helps everyone understand how data is organized and used in a system. Think of it like a reference book that explains:

1. **What each piece of data means.**
2. **What type of data it is** (like a number, text, or date).
3. **What values are allowed** for that data.

Why is it Important?

It helps developers, database managers, and anyone working with data understand how to use it correctly. Without it, there could be confusion or mistakes in how data is stored or used.

Example: College Admission Process

Imagine a **college admission system** where students apply, and the college collects data to decide whether to accept them. The data dictionary will explain how the data is stored and what kind of data is needed for each part of the application.

Here's what the **data dictionary** for this college admission process might look like in a simple table:

Data Element	Data Type	Description	Allowed Values
Student Name	Text	The full name of the student applying	Any combination of letters
Date of Birth	Date	The student’s date of birth	Any valid date (MM/DD/YYYY)
GPA	Decimal	The student's GPA	A decimal between 0.0 and 4.0
Application Status	Text	The status of the student's application	"Pending", "Accepted", "Rejected", "Waitlisted"
Course Preferences	List	List of courses the student prefers	A list of course names

Why Do We Need a Data Dictionary?

1. **Consistency:** It helps everyone working with the system follow the same rules, so the data is always the same format.
2. **Accuracy:** It prevents mistakes. For example, the data dictionary would make sure the GPA isn’t entered as a letter ("A+") but as a number (like 3.8).
3. **Clarity:** It makes it clear what kind of data is needed for each part of the process, which helps developers and others understand how to work with the system.

Why is a data dictionary important?

- It helps developers, data analysts, and anyone working with the data to understand the meaning and structure of the data.
- It reduces mistakes because everyone knows the rules and meanings of the data.
- It makes it easier to maintain, update, and improve systems over time.

Input and Output form design

In software engineering, **input and output form design** refers to the way we create and organize forms (screens or interfaces) for users to interact with a software application. These forms collect input from the user and display the results (output). The goal is to make the interaction clear, efficient, and user-friendly.

Input Form Design:

An input form is where users enter data. Good input form design focuses on how the user can easily understand what information is needed and how to provide it.

Key elements of input form design:

1. **Labels:** Clearly indicate what information is being requested. For example, “Name,” “Email Address,” or “Phone Number.”
2. **Input Fields:** These are where users type their information. They could be text boxes, checkboxes, radio buttons, dropdown menus, etc.
3. **Validation:** Ensures that the user’s input is correct. For instance, an email field should only accept text that looks like an email address.
4. **Help/Instructions:** Simple tips or guidance about how to fill out the form, like “Please enter your full name” or “Use numbers only.”
5. **Submit Button:** Allows users to send their data.

In software engineering, **validation** in input form design refers to the process of checking the data entered by users in a form to make sure it's correct, complete, and follows the required format. This helps prevent errors, ensures data consistency, and improves the user experience.

Why is validation important?

1. **Data accuracy:** Ensures the data entered is correct and in the right format.
2. **Security:** Helps prevent malicious or harmful data from being submitted.
3. **User experience:** Provides immediate feedback to users, guiding them to fix mistakes before submission.

Example:

Imagine you have a simple registration form on a website with the following fields:

- Name (Text input)
- Email (Text input)
- Age (Number input)

Now, let's look at the validation for each field:

1. Name:

- a) Validation could check that the field isn't left blank.
- b) It could also check that the name contains only letters (no numbers or special characters).
- c) Example: If the user enters John@123, the validation would catch it and show an error, "Name can only contain letters."

2. Email:

- a) Validation checks if the email entered follows the correct format (e.g., user@example.com).
- b) Example: If the user types john.com without the @, the validation would prompt an error, "Please enter a valid email address."

3. Age:

- a) Validation can ensure the age is a number and falls within a reasonable range (e.g., 18 to 100).
- b) Example: If the user enters abc instead of a number, the validation will show an error, "Please enter a valid age."

Example: Imagine you're designing a sign-up form for a website.

- **Name** (Input Field)
- **Email Address** (Input Field)
- **Password** (Input Field)
- **Submit** (Button)

Output Form Design:

An output form is where the software displays the results or feedback to the user after they have entered their data. It's all about presenting information in a clear, readable, and understandable way.

Key elements of output form design:

1. **Clear Results:** The output should be easy to read and understand.
2. **Formatting:** Information should be well-organized. Use headings, bullet points, or tables for complex data.
3. **Actionable Information:** If the user needs to take action based on the output, provide clear instructions (e.g., "Click here to continue" or "View your report below").
4. **Feedback:** Sometimes, the output needs to provide feedback on the process, like "Form submitted successfully" or "Error: Invalid input."

Example: After the user fills out the sign-up form, the output form could show:

- **Welcome, [User Name]!**
- **Your account has been successfully created.**
- **Check your email to verify your account.**

In software design, the output form is the part of the user interface that presents results or information to the user after they have interacted with the system. The key elements of output form design ensure that the user can easily interpret the data and take necessary actions. Let's break down these key elements:

Verification:

Code verification is the process used for checking the software code for errors introduced in the coding phase. The objective of code verification process is to check the software code in all aspects. This process includes checking the consistency of user requirements with the design phase. Note that code verification process does not concentrate on proving the correctness of programs. Instead, it verifies whether the software code has been translated according to the requirements of the user. The code verification techniques are classified into two categories, namely, dynamic and static. The dynamic technique is performed by executing some test data. The outputs of the program are tested to find errors in the software code. This technique follows the conventional approach for testing the software code.

In the static technique, the program is executed conceptually and without any data. In other words, the static technique does not use any traditional approach as used in the dynamic technique. Some of the commonly used static techniques are code reading, static analysis, symbolic execution, and code inspection and reviews.

Complexity analysis

It focuses on designs and actual code. They assume there is a direct correlation between design

complexity and design errors, and code complexity and hidden defects. By identifying the properties of each that related to their complexity, we can identify those high-risk applications that either should be revised or subjected to additional testing. Complexity metrics helps to determine the number and type of tests needed to cover the design or coded logic. There are several accepted methods for measuring complexity, most of which can be calculated by using automated tools.

Coding standards

The coding is the process of translating software design specification into source code through specific programming language. Coding is done by the coder or programmers who are independent people than the designer. Efficient coding reduce cost of testing and maintenance.

Coding standards are the set of guidelines or norms that are globally set by various software development organizations and these guidelines should be followed by all the developers to produce a good quality software.

Benefits of coding standards-

- Code understanding becomes easier
- Entire code of the whole software has same format and style
- The dependency of code on a particular developer is reduced
- It helps in code reuse and to detect error easily in early phase.
- Increase the efficiency of the software and reduces the development time.

➤ General coding standards

1. Standard headers for different modules:

For better understanding and maintenance of the code, the header of different modules should follow some standard format and information. The header format must contain below things:

- ✓ Name of the module
- ✓ data of creation, date of approval,
- ✓ developer's or author's name, synopsis,
- ✓ description of the module,
- ✓ description of the variables and functions defined in the module,

2. Naming conventions for local variables, global variables, constants and functions:

Meaningful and understandable variables name helps anyone to understand the reason of using it.

- ✓ Local variables should be named starting with small letter (e.g. local Data)
- ✓ Global variables names should start with a capital letter (e.g. Global Data).
- ✓ Constant names should be formed using capital letters only (e.g. CONSDATA).
- ✓ It is better to avoid the use of digits in variable names.
- ✓ The names of the function should be written in small letters.
- ✓ The name of the function must describe the reason of using the function clearly and briefly.

3. Indentation:

Proper indentation is very important to increase the readability of the code. For making the code readable, programmers should use White spaces properly. Some of the spacing conventions are given below:

There must be a space after giving a comma between two function arguments.

- ✓ Each nested block should be properly indented and spaced.
- ✓ Proper Indentation should be at the beginning and end of each block in the program.
- ✓ All braces should start from a new line

4. Error return values and exception handling conventions:

All functions that encountering an error condition should either return a 0 or 1 for simplifying the debugging.

5. Avoid using a coding style that is too difficult to understand:

Code should be easily understandable. The complex code makes maintenance and debugging

difficult and expensive.

6. Avoid using an identifier for multiple purposes:

Each variable should be given a descriptive and meaningful name indicating the reason behind using it. This is not possible if an identifier is used for multiple purposes and thus it can lead to confusion to the reader.

7. Code should be well documented:

The code should be properly commented for understanding easily. Comments regarding the statements increase the understandability of the code.

8. Length of functions should not be very large:

Lengthy functions are very difficult to understand. That's why functions should be small enough carry out small work and lengthy functions should be broken into small ones for completing small tasks.

9. Try not to use GOTO statement:

GOTO statement makes the program unstructured, thus it reduces the understandability of the program and also debugging becomes difficult.

Size Measures (Size Estimation): Two techniques are used to estimate project size.

1. Lines of code or LOC
2. Function point

1. Lines of code or LOC

LOC count the total number of lines of statements or instructions written in the source code of software project. LOC is used to predict the amount of effort that will be required to develop a program as well as estimate programming productivity. It is also used for calculating productivity of programmers.

- Count in LOC-

Declarations, Actual Code including logic and computation.

```
Ex:      for (i = 0; i < 100; i += 1)
        {
            Printf ("hello");
        } /* How many lines of code is this? */
```

- 4 Physical Lines of Code (LOC)
- 2 Logical Line of Code (LLOC)
- 1 comment line

- Don't count in LOC-

Blank Lines: used to improve the readability of code

Comments: used to help in code understanding as well as maintenance

```
Ex: for (i = 0; i < 100; i += 1)
    printf("hello"); /* How many lines of code is this?
```

The units of LOC are:

- KLOC- Thousand lines of code
- NLOC- Non comment lines of code
- KDSI- Thousands of delivered source instruction

Advantages:

- Estimation is closer to developer's perspective.
- Simple to use.

Disadvantages:

- It is language dependent
- No proper industry standard exist for this technique.
- It is difficult to estimate the size using this technique in early stages of project.

2. Functional Point (FP) Analysis

Allan J. Albrecht initially developed function Point Analysis in 1979 at IBM and it has been further modified by the International Function Point Users Group (IFPUG). The Function Point Analysis technique is used to analyze the functionality delivered by software to user. FPA is measuring software productivity and the cost associated with the development and maintenance.

The units of FPA: Unadjusted Function Point (UFP)

➤ Objectives of FPA

1. To measure functionality that the user requests and receives.
2. To measure software development and maintenance independently of technology used for implementation.
3. It should be simple enough to minimize the overhead of the measurement process.
4. It should be a consistent measure among various projects and organizations.

Types of FPA (Measurements Parameters of FPA):

1. Transactional Functional Type

- **External inputs**- data or control inputs (input Screen and files, tables, forms, messages, etc.) to the system
- **External outputs**- data or control outputs from the system (Output screens and reports)
- **External inquiries**- I/O queries which require a response (prompts, interrupts, calls, etc.)

2. Data Functional Type –

- **External interfaces** - libraries or programs which are passed into and out of the system (I/O routines, sorting procedures, math libraries, run-time libraries, etc.)
- **Internal data files**- groupings of data stored internally in the system (entities, internal control files, Databases and directories)

Weights of 5-FP Attributes

Measurement Parameter	Simple	Average	Complex
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal Logical Files (ILF)	7	10	15
External Interfaces (EIF)	5	7	10

*The functional complexities are multiplied with the corresponding weights against each function, and the values are added up to determine the UFP (Unadjusted Func

Function Point (FP)= Sum of Function point Count * CAF

Where Function Point is obtained from the above Table.

$$CAF = [0.65 + 0.01 * \sum(f_i)]$$

$\sum(f_i)$ is the sum of all 14 questionnaires scale(0 to 5) shows the **complexity adjustment factor-CAF** (where i ranges from 1 to 14).

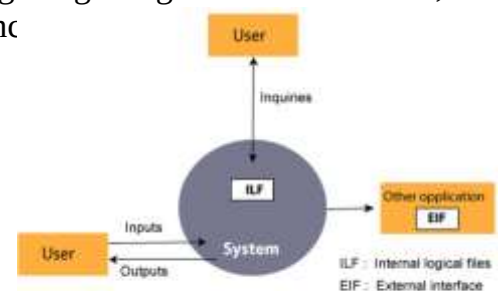
Example: Compute the function point, productivity, documentation and cost per function for the following data:

1. Number of user inputs = 24
2. Number of user outputs = 46
3. Number of inquiries = 8
4. Number of files = 4
5. Number of external interfaces = 2

Various processing complexity factors are: 4, 1, 0, 3, 3, 5, 4, 4, 3, 3, 2, 2, 4, 5.

Solution:

Measurement Parameter	Count	Weighting factor
-----------------------	-------	------------------



FPA's Functional Units System

Number of external inputs (EI)	24	*	4 = 96
Number of external outputs (EO)	46	*	4 = 184
Number of external inquiries (EQ)	8	*	6 = 48
Number of internal files (ILF)	4	*	10 = 40
Number of external interfaces (EIF)	2	*	5 = 10
Count-total →			378

So sum of all f_i ($i \leftarrow 1$ to 14) = $4 + 1 + 0 + 3 + 3 + 5 + 4 + 4 + 3 + 3 + 2 + 2 + 4 + 5 = 43$

$$\begin{aligned}
 FP &= \text{Count-total} * [0.65 + 0.01 * \sum(f_i)] \\
 &= 378 * [0.65 + 0.01 * 43] \\
 &= 378 * [0.65 + 0.43] = 378 * 1.08 = \mathbf{408}
 \end{aligned}$$

Advantages:

- Used in the early stages of project planning.
- Depending on the programming language.
- Used to compare different projects and technologies (database, language etc).

Disadvantages:

- It is not good for real time systems and embedded systems.
- Many cost estimation models like COCOMO uses LOC and FPC.

Differentiate between FP and LOC

FP	LOC
1. FP is specification based.	1. LOC is an analogy based.
2. FP is language independent.	2. LOC is language dependent.
3. FP is user-oriented.	3. LOC is design-oriented.
4. It is extendible to LOC.	4. It is convertible to FP (backfiring)

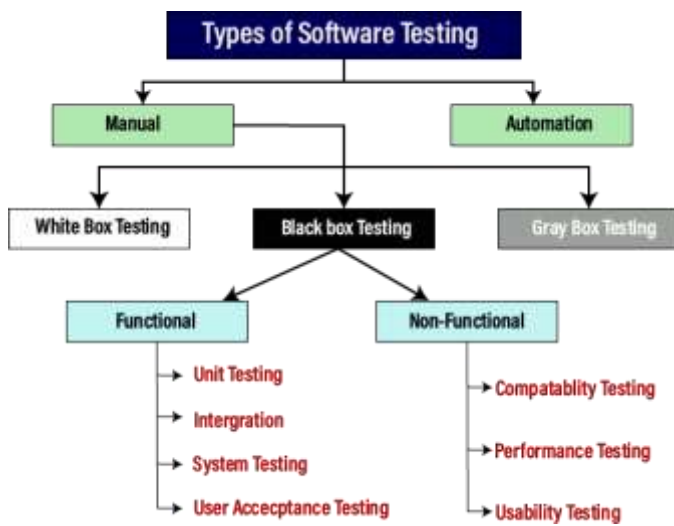
Testing Fundamentals: Testing is a group of techniques to determine the correctness of the application under the predefined conditions and data. It is a process of identifying the correctness of software by considering its all attributes (Reliability, Scalability, Portability, Re-usability, and Usability) and evaluating the execution of software components to find the software bugs or errors or defects.

Needs of Testing:

- It measures the quality of the software.
- Gives confidence in the quality.
- Reduces the overall level of risks.
- It Helps measure the reliability of the software.
- Verifies and validates user requirements.

The three main goals of Testing are:

- Defect Detection: Find defects / bugs in the software during all stages of its development.
- Defect Prevention: help anticipate and prevent defects from occurring at later stages of development or from recurring in the future.
- User Satisfaction: Ensure customers / users are satisfied that their requirements (explicit or implicit) are met.



Effort estimation in software engineering refers to the process of predicting the amount of work (effort) required to complete a software project or task. This helps project managers plan resources, timelines, and costs effectively.

Let's explore some **common estimation methods** using the example of an **online college admission process**.

1. Expert Judgment

What it is: This method involves asking experienced developers or experts to give their best estimate based on their previous experience with similar projects.

How it works:

- The team of experts, based on their knowledge of building similar systems, would estimate how long it will take to build the **online college admission process**.

Example: For the task of **account creation** (where students register on the website), the expert might say, "In my experience, it usually takes around 5 hours to create the registration page and set up the database." Based on this, they would estimate the effort.

Estimate: 5 hours for account creation.

2. Analogous Estimation

What it is: This method uses data from past, similar projects to make estimates. It's based on comparing the new project with previous ones.

How it works:

- If you've built a similar online system before (like a **student portal**), you can use that as a reference to estimate the effort for the **college admission system**.

Example: If you previously built an online **student registration system** that took 30 hours to complete, and the **admission system** is quite similar, you could estimate that the **college admission system** will take a similar amount of time.

Estimate: 30 hours (based on the previous project's effort).

3. Parametric Estimation

What it is: This method uses mathematical formulas based on certain parameters (like lines of code, number of features, etc.) to predict effort.

How it works:

- You identify a measurable unit of work, such as the **number of features** or **number of forms** in the admission system, and use a formula to estimate effort.

Example: For the **online college admission system**, let's say each feature (like registration, document upload, payment system, etc.) takes an average of **5 hours** to develop. If there are 6 features, the total effort is calculated as:

Estimate: 6 features × 5 hours = 30 hours.

4. Function Point Estimation

What it is: This method estimates effort based on the "function points" of a system. Function points are units that measure the functionality the system will provide, such as inputs, outputs, user interactions, etc.

How it works:

- You identify the **function points** in the system, such as input forms, user interactions, and outputs, and assign a weight to each point based on its complexity (simple, average, or complex).

Example: For the **online admission system**, the function points could be:

- **User login** (simple): 2 points.
- **Personal info form** (average): 3 points.
- **Document upload** (complex): 5 points.
- **Payment system** (complex): 6 points.

Now, you assign a time estimate based on the function points, for example:

- **2 points = 5 hours**
- **3 points = 8 hours**
- **5 points = 12 hours**
- **6 points = 15 hours**

Estimate: Total = 5 + 8 + 12 + 15 = 40 hours.

Cost estimation in software engineering is the process of predicting how much time, effort, and money will be needed to complete a software project. It's like creating a budget for building a software product.

Here's how it generally works:

1. **Determine the Scope:** Understand what the software needs to do. This involves gathering requirements and defining the features.

2. **Break Down the Work:** Divide the project into smaller tasks, like design, coding, testing, and deployment. This helps in understanding the amount of work involved.
3. **Estimate Resources:** For each task, estimate how many hours or days it will take. Consider the skills of the team, the complexity of the work, and the tools needed.
4. **Factor in Risks and Uncertainties:** Sometimes things don't go as planned. You need to consider possible risks (e.g., changes in requirements, technical difficulties) that could affect the cost.
5. **Calculate the Total Cost:** Add up all the hours or days estimated for each task, and multiply by the cost of labor (the hourly or daily rate of the developers).
6. **Adjust for Buffers:** Add some extra time or budget (called "contingency") in case there are unexpected issues.

By doing this, software teams can create a budget or a cost plan to make sure they stay on track and don't run out of resources before the project is completed.

The goal is to make an informed guess, knowing that the estimate might change as the project progresses.

Creating an E-commerce Website

- **Scope:**
 - Features:
 - Product listing page
 - Shopping cart
 - Checkout and payment integration
- **Break Down the Work:**
 - Design the website (5 days)
 - Code for product listing page (6 days)
 - Code for shopping cart (7 days)
 - Code for checkout/payment (8 days)
 - Test the website (4 days)
- **Estimate Resources:**
 - Total days needed for tasks:
 $5 \text{ (design)} + 6 \text{ (product page)} + 7 \text{ (cart)} + 8 \text{ (checkout)} + 4 \text{ (test)} = \mathbf{30 \text{ days}}$
- **Factor in Risks:**
 - Add 3 extra days for any unexpected challenges (e.g., payment gateway issues).
So, $30 \text{ days} + 3 \text{ days} = \mathbf{33 \text{ days}}$
- **Estimate the Budget:**
 - Developer costs \$200 per day: $33 \text{ days} \times \$200/\text{day} = \mathbf{\$6,600}$

Final Cost Estimate:

Creating an e-commerce website will cost around **\$6,600** and take about **33 days**.

Software Implementation and maintenance:

Systems implementation is the construction of the new system. Implementation is a process of ensuring that the information system is operational. It involves –

- Constructing a new system
- Constructing a new system from the existing one.

Implementation allows the users to take over its operation for use and evaluation. It involves training

the users to handle the system and plan for a smooth conversion.

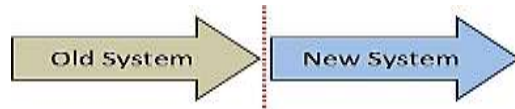
Conversion

The conversion is a process of replacing old existing system by new one. The all the data of old format is converted into new format.

➤ Conversion Methods-

1. Direct Changeover Conversion

An old system is directly replaced by the new system. It is the most risky conversion. It may be necessary when time is very short. It is also called crash conversion. The new system replaces entire existing system. The company stops using their old system and start using the new system.

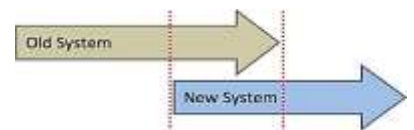


Example: a super market closes on a Friday; install all the software and hardware and re-opens on Monday with the new system in place.

Advantages	Disadvantages
▪ It is less costly as it is a direct change over ▪ It is not very time consuming as once the old system has stopped being Used the new system is immediately set up.	▪ Employees have to be trained before the implementation ▪ All the data will be lost if the new system fails and this will affect the whole organization. ▪ It is very difficult to detect small errors in the new system

2. Parallel Conversion

A type of conversion in which both new and old systems operate together for some of the time is called **parallel conversion**. It is the safest type of conversion. The results of both systems are compared. If the new system is successful then stop using the old system.



Example-an office prepares their payroll manually as well as using the payroll preparation system

Advantages	Disadvantages
▪ If the new system fails there will be no data lose because we have old system data ▪ Employees must be given training before implementation ▪ Small minor errors can be easily detected. ▪ Fixing any problems with the new system before ending previous system	▪ same work twice ▪ Delay in service, Customers unhappy. ▪ costly ▪ very time consuming

3. Phase-In Method

A type of conversion in which individual components of new system are implementation one by one is called phased conversion. Phased-In method introduces the new system to one department at a time.



Advantage	Disadvantages
▪ If the new system fails entire system is not affected ▪ More users friendly. ▪ Employees can be given training during the process of implementation. ▪ system is tested at every stage, very little chance of error	▪ Time consuming to implement. ▪ Costly

4. Pilot Approach

In this conversion, one part of the organization uses the new system and the rest of the organization uses the old system. When one part of the organization is satisfied with the new system, the rest of the organization can start using it.



Advantage	Disadvantage
▪ Risk is reduced ▪ Allows the organization to see whether the new system will meet the organizations needs in one department/location before using it throughout the entire organization	▪ It takes long time to run the pilot, which leads to higher cost and time taken up by staff to evaluate the system

Maintenance:

When new system is installed, the periodic maintenance is needed to restore system performance in original condition. System maintenance is an ongoing activity, which covers a wide variety of activities, including removing program and design errors, updating documentation and test data and updating user support.

Maintenance means restoring system to its original conditions. Thus, maintenance changes the existing system, adding new features, enhance and modifying the code.

There are various reasons for software maintenance listed below:

1. User requirements may changes.
2. Some problems occurs in the system or modules.
3. Some improvements required in the system efficiency and output.
4. Changes in hardware or software environment.
5. To modify the internal components of the system.

➤ Types of Maintenance

Corrective Maintenance	Repair design and programming errors (bugs)
Adaptive Maintenance	Adapting the change in environment (different computers or Different operating systems).
Perfective Maintenance	enhancing the system to meet the new requirements
Preventive Maintenance	changes made to improve future maintainability

Corrective maintenance: corrective maintenance is done to correct the detected problems or errors in different phases of the software system. The corrective maintenance is performed to restore or recover a system to a stable working condition after any performance failure occurs. This failure or error may be a logical error, coding error or may be any kind of design error.

This maintenance procedure is also known as fixing. Fixing is done in a short amount of time.

Adaptive maintenance: Adaptive maintenance basically deals with changing in program functions. It is performed when there are some external changes in the environment occurs. These external changes may occur due to any hardware-software technology changes or any government rule, business policy changes.

For example, a particular software was developed to calculate the taxes on profits of a company after deducting the dividend on equity shares. But after some period, the government has issued a new rule then software should be updated and the changes should be made in a particular function.

Perfective maintenance: Perfective maintenance is done to perfect level. It's basically an existing software system to meet user's new requirements. These maintenance are done by upgrading and modifying the software modules.

For example, in a sub-office, the official data were sent to headquarters via magnetic media. But after the sub-offices and headquarters are linked via a leased line, now they can use these leased lines to interchange data to headquarters. But in this situation, an existing software is needed to be enhanced to send the data via the leased line. Perfective maintenance is to be done to the existing software to enhance its performance but it is very time consuming and very costly. Software modification is simply the system level up-gradation by rewriting the programs.

Preventive maintenance: It is a kind of preventing a system being out of date. By applying preventive maintenance we recover a working software system from entering into an obsolete state. Preventive maintenance follows the two important concepts of software engineering – re- engineering & reverse engineering. The concept of re-engineering and reverse engineering includes the tasks like – optimization of existing code, updating the software documentation or sometimes re- structuring the software modules. Using these two concept, an old software having some old features and technology is re-engineered with some really new technology – which is now up to date and can produce great performance than the old system.

The preventive maintenance also makes the current system easier for the future – So that in future the maintenance can be easily implemented and the system can adopt the future maintenance in less difficulty.

Construction of the System: The planning about the modules of the system is done in the system designing system. The Process of Assembling the system is called as Construction of the System.

There are Two types of System Constructions: 1) Traditional Approaches

2) Incremental Approaches

1) Traditional Approaches: There are following steps are used in Traditional Approaches.

1.Unit Testing

2.Subsystem Testing

3.System Integration

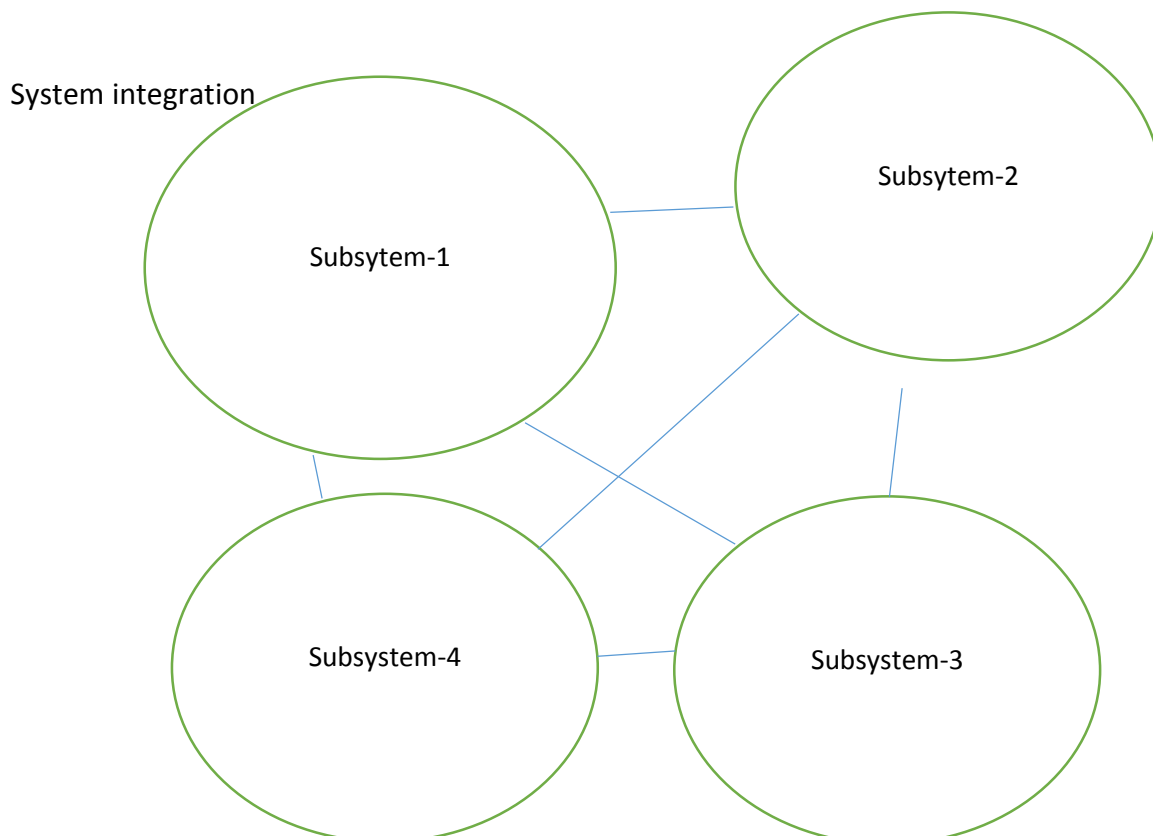
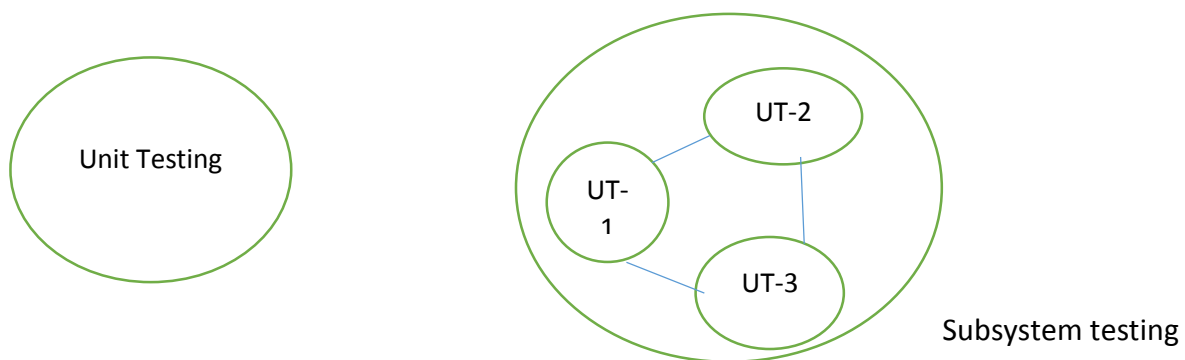
4.System Disintegration

1) Unit Testing: In this step we write test and Debug each module of system individually.

2) Subsystem Testing: In this step all the units of subsystem are tested and debugged as well as interface between units are tested. All subsystems are tested.

3) System Integration: In this step combined all subsystems to form a whole system.

4) System Disintegration: If any problem is occurred during the interfaces of system or subsystem then it is disintegrated or debugged again.



In this traditional model each and every model tested independently. As well as each subsystem is tested independently. Final subsystem is integrated and interfaces between subsystems also tested. These interfaces are very crucial but they are tested very late. So, if any errors occurred in these interfaces are difficult to remove. So, to removing these errors are time consuming.

So, in these situations we couldn't remove the errors or tested the software we cannot deliver the software to customer and not able to complete the deadline this is called as eleventh disaster.

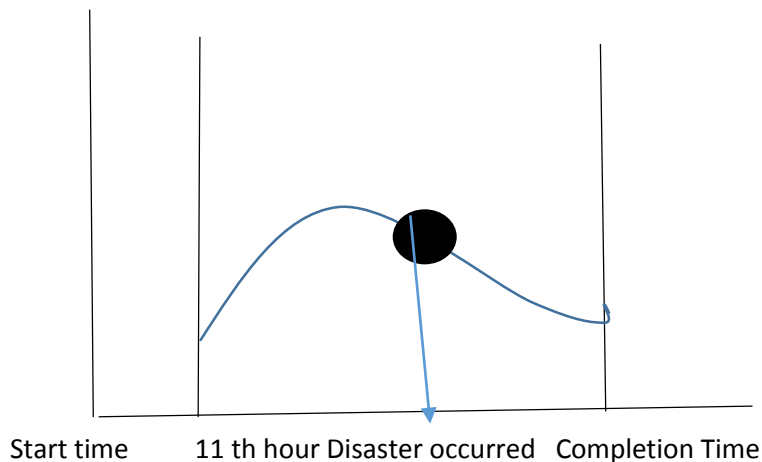
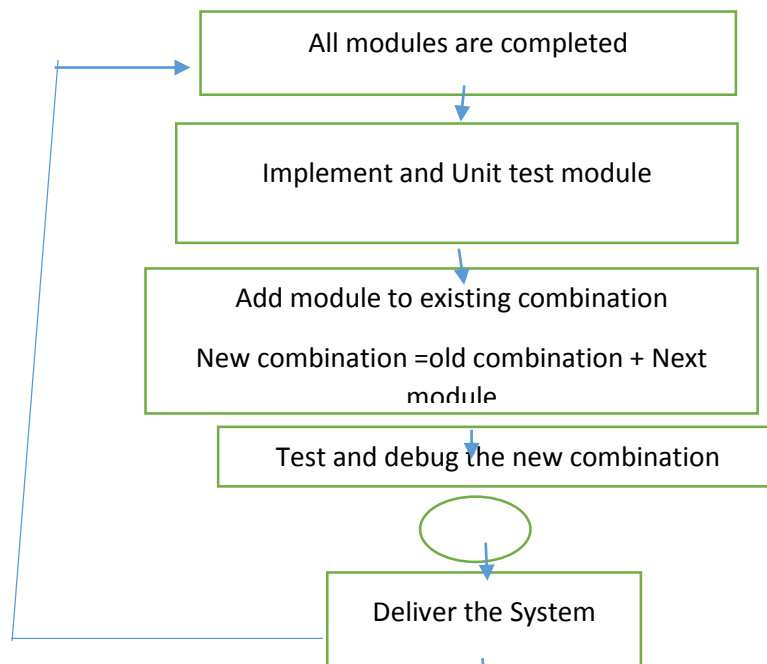


Fig. Testing and Debugging in Traditional Approach

- **Advantages:** This type of testing requires very little or no planning.
- All the modules are completed before the integration testing starts.
- **Disadvantages:**
 - Since all the modules are tested together chances of failure increases.
 - Difficult to find the root cause of the failure if failure occurs.
 - Difficult to detach the modules if any error occurs.
 - Time consuming process.

2) Incremental Approach: As compared to traditional approaches this is more disciplined module for system implementation. Fig. Flowchart for Incremental Approach



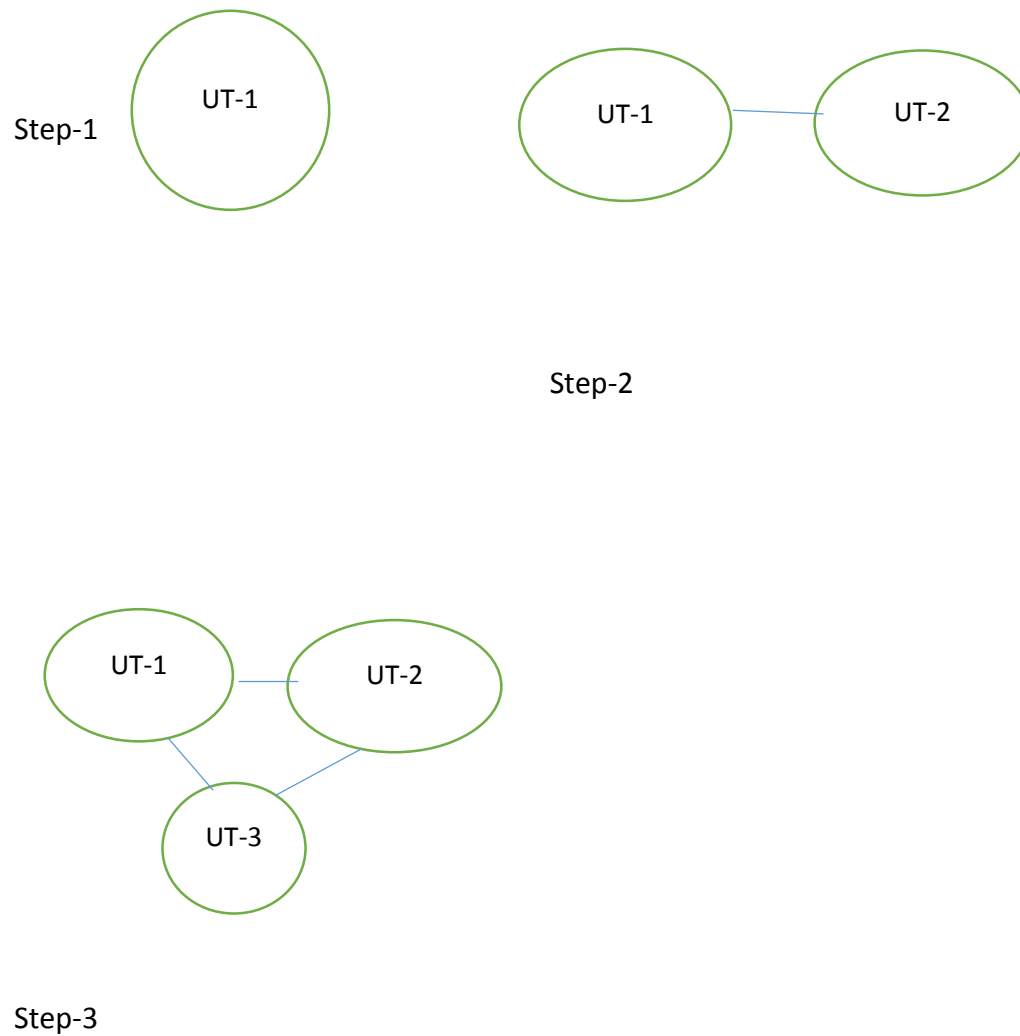


Fig. Steps in Incremental Approach

_In step 1 1st Module is Unit tested implemented.

-The Next module is unit tested and added with last module.

-This process goes on till the module are completed

From step-2 the interface testing is stored and it goes on till the last step (i.e. interface on second stage and it is tested till last module is completed).

-If any errors are occurred in any steps, it is removing then addition of new modules into existing combination is done. So, in this approach interface are tested early. But in traditional approach errors are found in last step

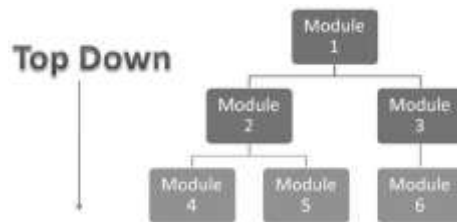
Incremental approaches done by using following methods.

1) **Top-down incremental Implementation:** In these approaches Top module is implemented first then the next module is implemented.

-In this approach the co-ordinate module is implemented first.

-Most important interfaces are present in co-ordinate modules, so they are tested early and errors also removing early.

- ◆ - For this implementation stubs are used as dummy modules. Stubs are used in Top-down testing approach. Stubs help create the interface between lower level modules which are not available or developed.



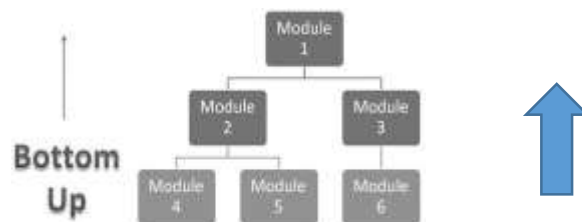
Example

- Testing will start from module 1 then proceed to module 2, module 3 etc.
- In top-down approach only module 1 is tested in isolation rest all other modules are integrated one by one and tested.
- Consider Module 2 is not developed yet so in order to test module 1 we need module 2, so here use stubs.
- So, in place of module 2 stubs are placed. This stub will accept input from module 1 and returns the result. Stubs can be created by tester.

2) **Bottom-up incremental Implementation:** In these approaches bottom-up module is implemented first then the next module is implemented.

-In these approaches the drivers are used as dummy modules.

- ◆ Drivers are used in Bottom-up testing approach. Drivers help create the interface between top level modules which are not developed.



Example

- Testing will start from module 4, module 5 and module 6 and this process will continue till modules are covered i.e. the process will come to halt once module 1 gets tested.

Top- Down Approach	Bottom-UP Approach
Top-Down testing conducted from main-Module to sub module.	Bottom-Up testing conducted from sub Module to main module.
If sub module is not developed a temporary Program called Stub is used to simulate the sub module.	If main module is not developed a temporary Program called Driver is used to simulate the main module.
Top -Down testing good if major flaws occur Toward the top of the program.	Bottom-Up Testing good if major flaws occur Toward the bottom of the program.
In this test condition difficult to create.	In this test condition easy to create.
Observation of test output is more difficult.	Observation of test output is easier.

3) Sandwich incremental Implementation: In this approach Top down and bottom-up modules are used.

-Some bottom and some top modules are implemented in these approaches.

-Both stubs and drivers are used as dummy modules for implementation in this approach.

Fig.Sandwich Modules

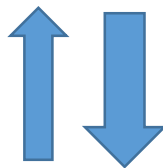
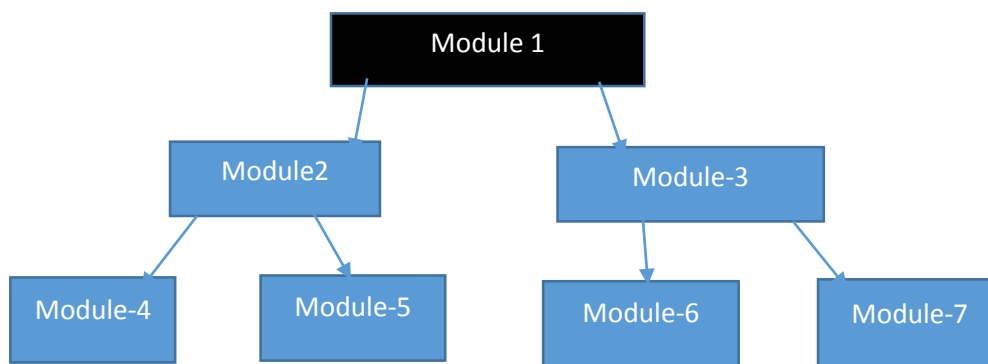


Fig.Hirarchical structure of modules with incremental Implementation Direction



Advantages of Incremental Approaches:

1. Crucial Interfaces are tested early so there is less chance of 11th disaster.
2. Delivery of system in fixed time.
3. The feedback is given to customer each and every step.
4. Versions of system are created and given to customer.
5. It saves the time because coding and testing done simultaneously.